

1.mcp_server01

mcp_server01 프로그램 개요

mcp_server01은 제조 설비 조회와 매뉴얼 검색 기능을 MCP Tool로 감싸는 Day5 1단계 교육용 예제입니다.

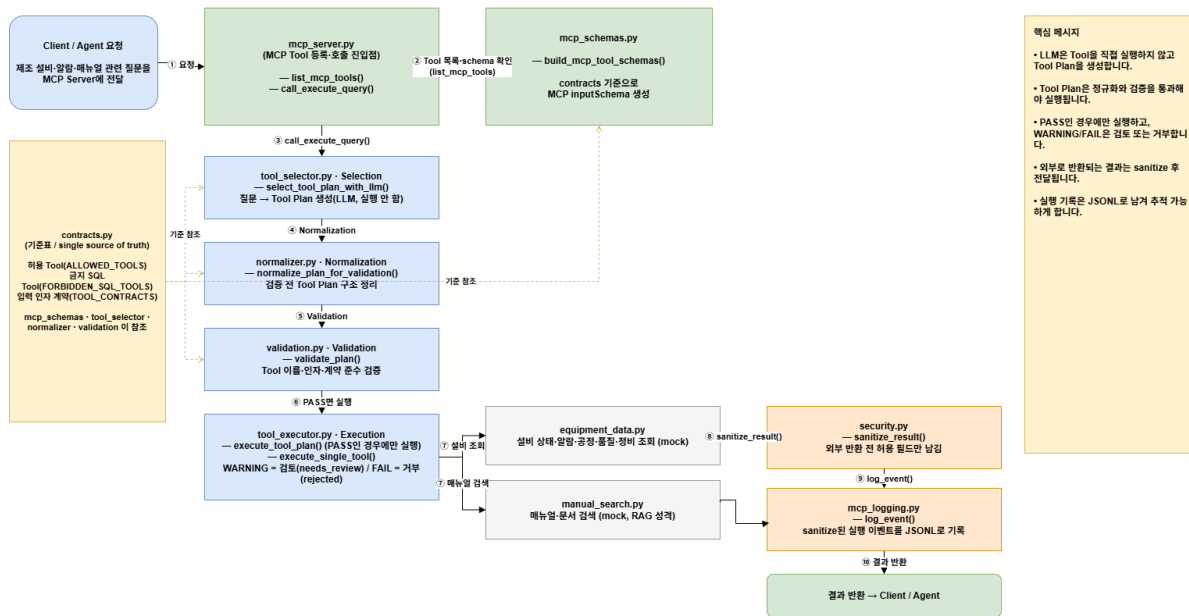
이 예제는 Agent가 내부 기능을 직접 호출하지 않고, MCP Server를 통해 표준화된 Tool로 접근하는 구조를 보여 줍니다.

핵심 흐름은 다음과 같습니다.

사용자 질문 → Tool Selection → Tool Plan 정규화 → Tool Contract 검증 → PASS인 경우에만 Tool 실행 → 결과
sanitize → 실행 기록 저장

mcp_server01 구조

핵심 파일과 핵심 함수 중심 구조



mcp_server01 전체 실행 흐름

실행 흐름

단계	처리 내용	비고
1	Client / Agent 요청	사용자가 제조 설비, 알람, 매뉴얼 관련 질문 입력
2	Tool 목록 확인	list_mcp_tools()로 호출 가능한 Tool과 입력 스키마 확인
3	Tool Selection	LLM이 사용자 질문을 Tool Plan으로 변환
4	정규화	Tool Plan을 검증 가능한 구조로 정리
5	검증	Tool 이름, 인자, 계약 준수 여부 확인
6	실행	PASS인 경우에만 Tool 실행
7	보안 반환	결과에서 허용된 필드만 남겨 반환
8	실행 기록	sanitize된 이벤트를 JSONL로 기록

정리

mcp_server01은 Tool을 바로 실행하지 않습니다. 먼저 선택하고, 정규화하고, 검증한 뒤, 통과한 요청만 실행합니다.

핵심 파일

중요도	파일명	역할	비고
상	mcp_server.py	MCP 서버 진입점	MCP Tool 등록과 Client 호출 진입 구조
상	contracts.py	Tool 계약 기준	허용 Tool, 금지 Tool, required / optional 인자 기준
상	tool_selector.py	Tool Plan 생성	LLM이 사용자 질문을 실행 계획으로 변환
상	tool_executor.py	검증 후 실행	PASS일 때만 Tool을 실행하는 게이트 구조
상	security.py	결과 보호	반환 결과에서 허용 필드만 남기는 sanitize 구조
중	mcp_schemas.py	MCP schema 생성	contracts 기준으로 Tool inputSchema 생성
중	equipment_data.py	설비 조회 mock	설비 상태, 알람, 품질, 정비 데이터 조회 예제
중	manual_search.py	매뉴얼 검색 mock	문서 근거 검색 Tool 예제

contracts.py는 계약, **tool_selector.py**는 선택, **tool_executor.py**는 실행, **security.py**는 보호를 담당합니다.

핵심 함수

함수명	파일명	역할
list_mcp_tools()	mcp_server.py	사용 가능한 Tool 목록과 스키마 반환
call_execute_query()	mcp_server.py	질문부터 실행까지 연결하는 end-to-end 진입점
select_tool_plan_with_llm()	tool_selector.py	LLM으로 Tool Plan 생성
execute_tool_plan()	tool_executor.py	정규화, 검증, 실행 여부 결정
validate_plan()	validation.py	Tool Plan의 계약 준수 여부 검증
execute_single_tool()	tool_executor.py	단일 Tool 실행 및 결과 처리
sanitize_result()	security.py	반환 결과에서 허용 필드만 유지
create_fastmcp_server()	mcp_server.py	FastMCP 객체 생성 및 Tool 등록

Tool Contract 기반 실행 제어

mcp_server01은 Tool 이름과 인자를 자유롭게 허용하지 않습니다. contracts.py에 정의된 계약을 기준으로 실행 가능 여부를 판단합니다.

구분	설명
허용 Tool	Agent가 호출할 수 있는 Tool 목록
금지 Tool	Text-to-SQL 계열처럼 실행을 차단할 Tool
required 인자	반드시 필요한 입력값
optional 인자	선택적으로 사용할 수 있는 입력값
validation 결과	PASS / WARNING / FAIL 판단

보안 게이트

보안 기준	설명
SQL Tool 차단	Text-to-SQL 계열 요청은 즉시 거부
Tool allowlist	허용된 Tool만 실행 가능
인자 제한	계약에 없는 인자 사용 방지
결과 sanitize	내부 데이터 전체가 외부로 나가지 않도록 제한
JSONL 로깅	실행 기록을 추적 가능하게 저장

Tool Contract는 단순한 명세가 아니라 **Agent**가 무엇을 호출할 수 있고, 어떤 조건에서 실행할 수 있는지 결정하는 운영 기준입니다.

2.mcp_server02

프로그램 개요

mcp_server02는 Day5 MCP Server 2단계 교육용 예제입니다. 1단계인 mcp_server01이 mock 기반 MCP Tool 서버 구조를 보여 주었다면, mcp_server02는 그 구조를 유지한 상태에서 search_manual Tool을 실제 RAG 검색 흐름으로 확장한 예제입니다.

즉, 외부에서 호출하는 MCP Tool 목록과 실행 흐름은 1단계와 거의 동일하게 유지하면서, 내부 검색 구현만 mock에서 RAG 기반 검색 구조로 교체합니다.

핵심 확장 흐름은 다음과 같습니다.

```
manual_search.py
→ rag_quality_adapter.py
→ rag_search/retrieval.py
→ rag_search/reranking.py
→ vector_db / Chroma DB
```

mcp_server02는 Ollama 임베딩과 Chroma 검색을 우선 사용하고, 검색 환경이 준비되지 않았거나 기술적 오류가 발생하면 mock 검색으로 fallback합니다. 이를 통해 수강생은 실제 RAG 연동 구조와 fallback 구조를 함께 이해할 수 있습니다.

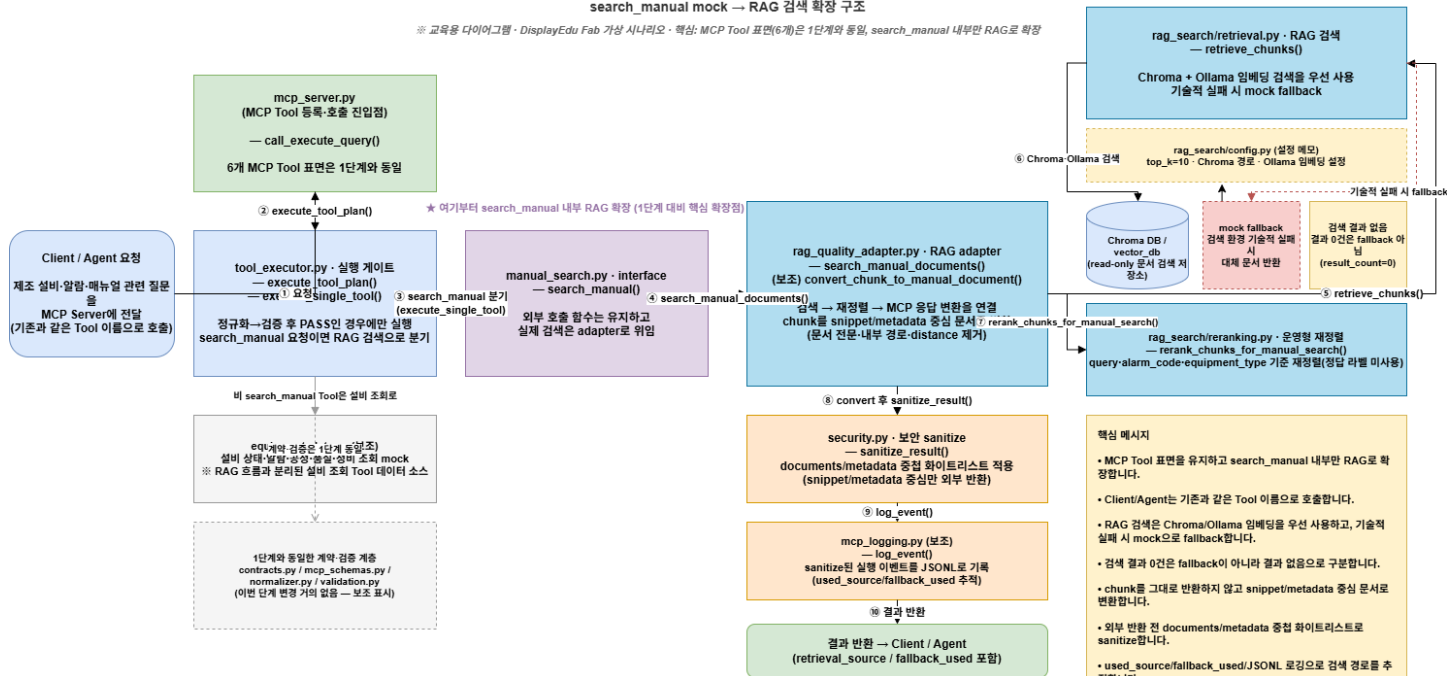
이 예제의 핵심은 다음 세 가지입니다.

1. MCP Tool 표면은 유지하고 내부 구현만 RAG로 확장
2. 검색 결과를 Agent가 사용할 수 있는 문서 근거 형태로 변환
3. 외부 반환 전 snippet과 metadata를 제한하여 보안 경계 유지

mcp_server02 구조

search_manual mock → RAG 검색 확장 구조

※ 교육용 다이어그램 · DisplayEdu Fab 가상 시나리오 · 핵심: MCP Tool 표현(6개)은 1단계와 동일, search_manual 내부만 RAG로 확장



보안-추적 게이트 (전 구간 공통)
Tool 표현 1단계 동일 | search_manual 내부만 RAG 확장 | 기술적 실패 시 mock fallback(점선) | 결과 0건=결과 없음 | 문서 전문-경로-distance 제거 | snippet/metadata 중첩 반환 | used_source-fallback_used 추적 | JSONL 로깅

mcp_server02 아키텍처

mcp_server02는 1단계의 MCP Tool 구조를 유지하면서, 내부 검색 구현을 mock 기반에서 실제 RAG 검색 흐름으로 확장한 아키텍처입니다.

01

1단계 구조 유지

외부에서 호출하는 MCP Tool 목록과 실행 방식은 mcp_server01과 거의 동일하게 유지합니다.

02

실제 검색으로 확장

search_manual Tool은 manual_search.py를 통해 RAG 검색 파이프로 연결됩니다.

03

Fallback 포함

Ollama 임베딩과 Chroma DB 검색이 가능하면 우선 사용하고, 실패 시 mock 검색으로 전환합니다.

핵심 흐름은 다음과 같습니다.

```
manual_search.py
→ rag_quality_adapter.py
→ rag_search/retrieval.py
→ rag_search/reranking.py
→ vector_db / Chroma DB
```

이 구조를 통해 수강생은 MCP Tool 표면은 단순하게 유지하면서도, 내부에서는 실제 RAG 검색과 결과 정제, 그리고 fallback 전략이 함께 동작하는 방식을 이해할 수 있습니다.

mcp_server01 대비 확장점

구분	mcp_server01	mcp_server02	비고
search_manual 구현	mock 문서 반환	RAG 검색 계층으로 확장	외부 Tool 이름은 유지하고 내부 검색 구현만 교체
검색 계층	별도 검색 계층 없음	rag_quality_adapter.py, rag_search 추가	Day4 RAG 평가 자산 중 검색-재정렬 흐름을 서버 구조에 연결
검색 방식	고정 mock 결과	Chroma / Ollama 임베딩 검색 우선	실제 검색과 mock fallback을 used_source, fallback_used로 구분
재정렬	없음	운영형 reranking 적용	정답 라벨이 아닌 query, alarm_code, equipment_type 기준으로 재정렬
보안 처리	top-level sanitize 중심	documents / metadata 중첩 화이트 리스트 적용	문서 전문, 내부 경로, distance 등 민감하거나 불필요한 필드 제거

정리하면, mcp_server02는 MCP Tool의 외부 계약을 유지하면서 내부 검색 기능을 RAG로 확장한 단계입니다.

이 구조는 현업에서 기존 Agent 호출 방식을 크게 바꾸지 않고 검색 품질을 개선할 수 있는 설계 방향을 보여 줍니다.

핵심 파일

파일명	역할	비고
mcp_server.py	MCP Tool 등록 및 호출 진입점	6개 Tool 등록 구조와 1단계와 동일한 Tool 표면 확인
manual_search.py	search_manual interface 계층	외부 호출은 유지하고 내부 검색 구현만 adapter로 위임
rag_quality_adapter.py	RAG 검색 연결 adapter	검색, 재정렬, MCP 응답 변환을 연결하는 핵심 계층
rag_search/retrieval.py	실제 검색 수행	Chroma / Ollama 임베딩 검색 우선, 실패 시 mock fallback
rag_search/reranking.py	검색 결과 재정렬	운영형 기준으로 검색 결과 순서를 재조정
security.py	검색 결과 sanitize	documents와 metadata 내부 필드까지 제한
tool_executor.py	검증 후 Tool 실행	search_manual 요청을 manual_search.py로 dispatch
rag_search/config.py	RAG 검색 설정	top_k, Chroma, Ollama 기본 설정 확인

핵심 정리

mcp_server02에서 가장 중요한 파일은 manual_search.py, rag_quality_adapter.py, rag_search/retrieval.py, rag_search/reranking.py입니다. 이 네 파일을 중심으로 보면 search_manual이 mock 검색에서 RAG 검색으로 어떻게 확장되었는지 파악할 수 있습니다.

핵심 함수

함수명	파일명	역할	비고
call_execute_query()	mcp_server.py	질문부터 Tool 실행까지 연결	1단계와 동일한 end-to-end 진입점
execute_tool_plan()	tool_executor.py	정규화·검증 후 실행 여부 결정	PASS인 경우에만 실행하는 게이트 구조
execute_single_tool()	tool_executor.py	단일 Tool 실행 및 dispatch	search_manual요청이 RAG 검색으로 넘어가는 분기점
search_manual()	manual_search.py	매뉴얼 검색 interface	외부 호출 함수는 유지하고 내부 구현만 RAG로 확장
search_manual_documents()	rag_quality_adapter.py	RAG 검색·재정렬·응답 변환 진입점	retrieval, reranking, 응답 변환을 연결
retrieve_chunks()	rag_search/retrieval.py	Chroma 기반 chunk 검색	실제 검색과 mock fallback을 구분
rerank_chunks_for_manual_search() ()	rag_search/reranking.py	검색 결과 운영형 재정렬	query, alarm_code, equipment_type 기준으로 가점
convert_chunk_to_manual_document() ()	rag_quality_adapter.py	chunk를 MCP 응답 문서로 변환	snippet, metadata 중심으로 반환 구조 정리
sanitize_result()	security.py	반환 결과 보안 처리	문서 전문, 경로, distance 등 불필요한 필드 제거

3.mcp_server03

프로그램 개요

mcp_server03은 Day5 MCP Server 3단계 교육용 예제입니다. 2단계인 mcp_server02가 search_manual Tool을 RAG 검색으로 확장했다면, mcp_server03은 설비 상태, 알람 이력, 공정 상태, 품질 지표, 정비 이력 조회 기능을 실제 PostgreSQL 읽기 전용 조회 구조로 확장한 단계입니다.

이 예제의 핵심은 Agent가 SQL을 직접 생성하지 않는다는 점입니다. LLM은 어떤 Tool을 호출할지 선택하고, 실제 DB 조회는 서버 내부의 고정 SQL 함수가 담당합니다.

전체 구조는 다음 흐름으로 이해할 수 있습니다.

Tool 실행 요청

→ equipment_data.py

→ repositories.py

→ db_access.py

→ PostgreSQL

DB 조회는 우선 PostgreSQL을 사용하고, DB 연결 실패 시에는 mock 데이터로 fallback합니다. 이때 data_source, fallback_used 값을 함께 남겨 실제 DB 조회인지, mock fallback인지 추적할 수 있게 구성되어 있습니다.

mcp_server03의 핵심 포인트는 다음 세 가지입니다.

1. LLM이 SQL을 만들지 않고 Tool별 고정 SQL로 DB를 안전하게 조회하는 구조
2. DB 연결 실패, 사용 오류, 조회 결과 없음의 차이를 구분하는 설계
3. 읽기 전용 DB 접근, SELECT 전용 방어, parameter binding, 결과 sanitize를 통한 보안 경계

이전 단계 대비 확장점

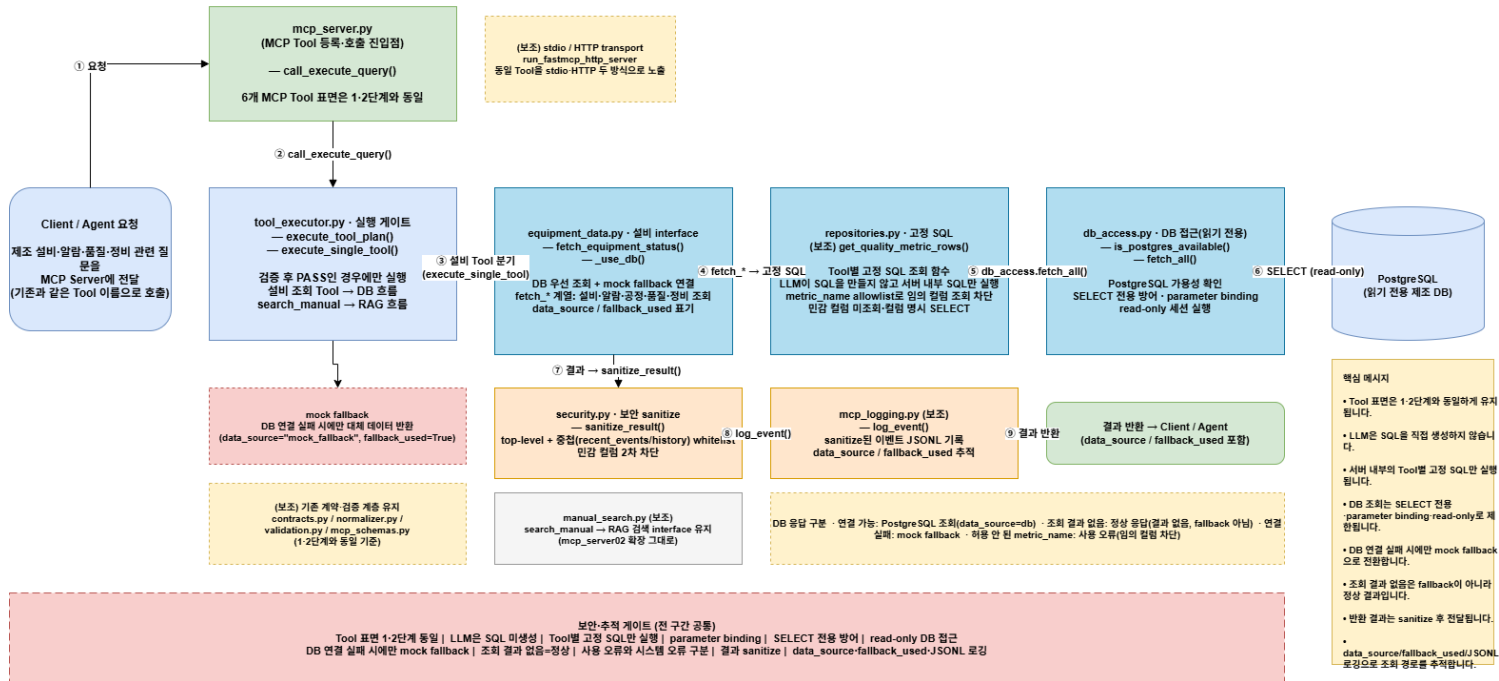
구분	mcp_server01	mcp_server02	mcp_server03	비고
설비 조회	mock 데이터	mock 데이터	PostgreSQL 읽기 전용 조회	설비·알람·공정·품질·정비 Tool이 실제 DB 조회 구조로 확장
RAG 검색	mock 검색	RAG 검색 확장	RAG 검색 유지	search_manual은 2단계 구조를 유지하고, DB 조회 축이 새로 확장
DB 계층	없음	없음	repositories.py, db_access.py 추가	고정 SQL 계층과 DB 접근 계층을 분리
fallback	mock 중심	RAG 검색 실패 시 mock fallback	DB 연결 실패 시 mock fallback	실제 DB 조회와 fallback 결과를 data_source, fallback_used로 구분
전송 방식	stdio 중심	stdio 중심	stdio + HTTP transport	동일 Tool을 stdio와 HTTP 방식으로 노출 가능

핵심은 Tool 표면은 유지하고 내부 데이터 소스만 단계적으로 확장한다는 점입니다. mcp_server03은 RAG 검색 구조에 더해, 실제 DB 조회 구조를 MCP Tool 서버 안에 안전하게 넣는 방법을 보여 줍니다.

mcp_server03 구조

PostgreSQL 읽기 전용 조회 확장 구조

※ 교육을 다이어그램 - DisplayEdu Fab 가상 사나리오 - 핵심: Tool 표현은 1-2단계와 동일, 예비 조회 Tool만 mock → PostgreSQL 읽기 전용 조회로 확장



파일 설명

파일명	역할	비고
db_access.py	PostgreSQL 읽기 전용 접근 계층	SELECT 전용 방어, parameter binding, read-only 세션을 통해 DB 접근을 제한
repositories.py	Tool별 고정 SQL 조회 함수	LLM이 SQL을 만들지 않고, 서버 내부의 검증된 SQL만 실행
equipment_data.py	설비 데이터 interface	DB 우선 조회와 mock fallback을 연결하는 중간 계층
security.py	결과 sanitize 계층	DB 응답의 top-level 및 중첩 필드를 제한하여 민감 정보 노출 방지
mcp_server.py	MCP 서버 진입점과 전송 계층	6개 MCP Tool 표면 유지, HTTP transport 추가
tool_executor.py	Tool 실행 게이트	검증된 Tool Plan만 실행하고, 설비 Tool과 RAG Tool로 분기
manual_search.py	RAG 검색 interface	2단계의 search_manual RAG 구조 유지
contracts.py	Tool 계약 기준	1·2단계와 동일한 Tool 계약을 유지

핵심 정리

mcp_server03에서 가장 중요한 확장 파일은 equipment_data.py, repositories.py, db_access.py입니다. 이 세 파일을 중심으로 보면 MCP Tool 요청이 실제 PostgreSQL 읽기 전용 조회로 연결되는 구조를 이해할 수 있습니다.

핵심 함수

함수명	파일명	역할	비고
call_execute_query()	mcp_server.py	질문부터 Tool 실행까지 연결	1·2단계와 동일한 end-to-end 진입점
execute_tool_plan()	tool_executor.py	정규화·검증 후 실행 여부 결정	PASS인 경우에만 Tool 실행
execute_single_tool()	tool_executor.py	Tool별 실행 분기	설비 Tool은 DB 조회, search_manual은 RAG 검색으로 분기
fetch_equipment_status()	equipment_data.py	설비 상태 조회 대표 함수	DB 우선 조회와 mock fallback 구조를 설명하기 좋은 함수
_use_db()	equipment_data.py	DB 사용 여부 판단	강제 mock 여부와 PostgreSQL 가용성을 기준으로 분기
is_postgres_available()	db_access.py	PostgreSQL 연결 가능 여부 확인	매 호출마다 긴 대기를 막기 위한 가용성 판단
get_quality_metric_rows()	repositories.py	품질 지표 고정 SQL 조회	metric_name allowlist로 임의 컬럼 조회를 차단
fetch_all()	db_access.py	SELECT 실행 및 결과 dict 변환	parameter binding, SELECT 전용 방어, read-only 실행 구조
sanitize_result()	security.py	반환 결과 필드 제한	DB 결과의 중첩 리스트 내부 key까지 제한
log_event()	mcp_logging.py	실행 이벤트 JSONL 기록	data_source, fallback_used 추적 근거 저장

4.mcp_client

개요

Day5의 mcp_client01~04는 앞 단계에서 만든 MCP Server의 Tool을 호출하는 쪽, 즉 MCP Client 관점을 학습하기 위한 예제입니다. MCP Server는 DB, RAG, Tool 실행 기능을 정의하고 외부에 노출합니다. 반면 MCP Client는 서버 내부 구현을 직접 알지 않고, Tool 이름과 입력값을 기준으로 Tool을 발견하고 호출합니다.

mcp_client01~04는 같은 서버 기능을 사용하되, 호출 방식이 단계적으로 확장됩니다.

구분	주요 역할	연결 방식	비고
mcp_client01	단일 Tool 호출과 기본 데모	direct import	Tool 발견, Tool 호출, 결과 요약의 기본 구조
mcp_client02	LangGraph 멀티 에이전트 구조	direct import	Planner, 여러 Agent, Evidence Check, 최종 종합 흐름
mcp_client03	실제 MCP transport 연결	stdio / http	direct 호출과 실제 MCP 연결 방식의 차이
mcp_client04	transport + LangGraph 통합	ToolCaller 추상화	supervisor 라우팅, 안전 분기, 병렬 DB Tool 호출

한 줄 정리

client01~04는 Tool을 호출하는 방식이 점진적으로 확장되는 구조입니다. Server 기능은 유지하고, Client 쪽 호출 방식만 직접 호출 → 그래프 → 실제 transport → 통합 실행으로 발전합니다.

mcp_client01 핵심 구조

mcp_client01은 MCP Client의 가장 기본 흐름을 보여 주는 예제입니다. mcp_server03의 call_mcp_tool, list_mcp_tools를 직접 import하여 Tool 목록 확인과 Tool 호출을 수행합니다. 실제 MCP transport는 사용하지 않으며, direct mode로 Tool 호출 구조를 먼저 이해하는 단계입니다.

핵심 파일

파일명	역할	비고
client_app.py	CLI 진입점과 데모 실행	Tool 목록 확인, RAG 검색, DB 조회, 금지 Tool 거부 흐름
result_formatting.py	Tool 결과 요약	전체 dict를 그대로 출력하지 않고 필요한 정보만 요약
main.py	실행 위임	client_app.main()을 호출하는 보조 진입점

핵심 함수

함수명	역할	비고
run_demo()	6단계 데모 실행	Tool 목록 → RAG → DB → 금지 Tool 거부 흐름
_load_server_entrypoints()	서버 entrypoint 지연 import	Client가 서버 내부 구현이 아닌 entrypoint만 사용
run_single_tool()	단일 Tool 호출	CLI에서 지정한 Tool을 호출
summarize_tool_result()	결과 요약	정상 / rejected / data_source 정보를 보기 좋게 정리
safe_preview()	긴 값 미리보기	문서 전문이나 긴 문자열 노출 방지

mcp_client02 핵심 구조

mcp_client02는 direct mode 위에 LangGraph 멀티 에이전트 구조를 올린 예제입니다. Planner가 사용자 질문을 분석하고, 여러 Agent가 필요한 Tool 호출 여부를 판단한 뒤 Evidence Check와 Synthesis 단계로 결과를 정리합니다.

실제 transport는 아직 사용하지 않으며, Tool 호출은 direct wrapper를 통해 서버 endpoint로 전달됩니다.

핵심 파일

파일명	역할	비고
graph_nodes.py	Planner와 Agent Node 구현	need flag 기반 Tool 호출과 skip 처리
graph_builder.py	LangGraph 그래프 구성	fan-out / fan-in 구조 표현
graph_state.py	State 구조 정의	Agent 간 결과 충돌을 막는 State 설계
result_synthesis.py	최종 답변 생성	LLM 없이 템플릿 기반으로 안전하게 종합

핵심 함수

함수명	역할	비고
build_graph()	LangGraph 구성	Planner → Agent → Evidence → Synthesis 흐름
planner_node()	질문 분석	규칙 기반으로 필요한 Agent와 Tool 판단
_run_agent()	Agent별 Tool 호출	서버 Tool 호출 결과를 표준 구조로 변환
evidence_check_node()	근거 상태 확인	검색 결과와 mock fallback 여부 표시
synthesis_node()	최종 답변 생성	근거 기반 요약, 원인 단정 방지

mcp_client03 핵심 구조

mcp_client03은 실제 MCP transport를 사용하는 단계입니다. fastmcp.Client를 사용해 mcp_server03에 stdio 또는 http 방식으로 연결합니다. 이 단계에서는 LangGraph나 멀티 에이전트를 사용하지 않고, 실제 transport를 통해 단일 Tool을 호출하는 구조를 확인합니다.

핵심 파일

파일명	역할	비고
transport_stdio.py	stdio 기반 MCP 연결	서버 subprocess spawn, timeout 처리
transport_http.py	http 기반 MCP 연결	long-lived 서버 연결, RAG 호출 안정성
client_app.py	CLI 실행 분기	stdio / http 모드 선택
result_formatting.py	결과 요약	민감정보를 제외하고 요약 출력

핵심 함수

함수명	역할	비고
call_tool_stdio()	stdio로 Tool 호출	실제 MCP 메시지 기반 호출
list_tools_stdio()	stdio로 Tool 목록 확인	MCP Tool 발견 흐름
run_default_demo_stdio()	stdio 데모 실행	단계별 timeout 격리
call_tool_http()	http로 Tool 호출	서버 미기동 등 연결 실패 처리
_call_mcp()	timeout 적용 호출	무한 대기 방지

mcp_client04 핵심 구조

mcp_client04는 실제 transport와 LangGraph 라우팅을 통합한 단계입니다. supervisor가 사용자 질문을 분석하고, db / rag / safety / answer 흐름 중 하나로 라우팅합니다. 또한 ToolCaller 추상화를 통해 stdio, http, Fake caller를 교체할 수 있습니다. DB Tool 병렬 데모에서는 ThreadPoolExecutor를 사용해 실제 병렬 호출 구조를 확인할 수 있습니다.

핵심 파일

파일명	역할	비고
agents.py	supervisor와 역할 Agent 구현	규칙 기반 라우팅, safety / rag / db 분리
runner.py	실행 runner와 병렬 데모	CLI와 UI에서 함께 사용할 수 있는 실행 계층
mcp_tools.py	ToolCaller 추상화	stdio / http / Fake caller 교체 구조
graph.py	조건부 라우팅 그래프	supervisor 기반 단일 경로 선택

핵심 함수

함수명	역할	비고
supervisor_agent()	질문 라우팅	safety → tools → rag → db → answer 우선순위
db_agent()	DB Tool 선택 및 호출	질문을 안전한 조회 Tool로 연결
_pick_db_tool()	DB Tool 선택	위험 SQL 생성 없이 허용 Tool 선택
run_query()	그래프 실행	초기 State를 만들고 최종 State 반환
run_parallel_db_tools_demo()	병렬 DB Tool 호출	client02의 논리적 병렬과 실제 병렬 차이 설명

client01~04 비교표

비교 항목	mcp_client01	mcp_client02	mcp_client03	mcp_client04
주요 목적	기본 Tool 호출	LangGraph 멀티 에이전트	실제 MCP transport	transport + 그래프 통합
MCP 연결 방식	direct import	direct wrapper	stdio / http	stdio / http ToolCaller
Tool 호출 방식	call_mcp_tool 직접 호출	call_server_tool 경유	MCP wrapper Tool 호출	ToolCaller.call_tool()
Tool Selection	CLI 지정	규칙 기반 Planner	CLI 지정	규칙 기반 supervisor
LangGraph	없음	있음	없음	있음
Multi-Agent	없음	있음	없음	있음
Streamlit UI	확인되지 않음	확인되지 않음	확인되지 않음	폴더 내 확인되지 않음
병렬 구조	없음	논리적 병렬	없음	ThreadPool 실제 병렬
교육 포인트	호출 골격	그래프 구조	transport 차이	통합 라우팅과 병렬 실행

Day5 최종 통합 실습 목표

Day5 최종 통합 실습은 RAG 기반 기술 문서 검색과 MCP 기반 업무 Tool 호출을 결합하여 제조 업무형 AI Agent 흐름을 완성하는 단계입니다. 수강생은 사용자 질문이 입력된 뒤 Agent가 어떤 Tool을 선택하고, DB 조회와 문서 검색 결과를 어떻게 결합하며, 최종적으로 원인 후보, 조치 방안, 추가 확인 사항을 도출하는지 확인합니다.

실습 목표

1 통합 구조 확인

Agent, LangGraph, RAG, MCP, DB, Log가 하나의 흐름에서 어떻게 연결되는지 확인합니다.

2 Tool 호출 흐름 확인

Client가 Server 내부 구현을 직접 호출하지 않고 MCP Tool을 발견하고 호출하는 구조를 확인합니다.

3 DB + RAG 결합

설비, 알람, 품질, 정비 조회 결과와 기술 가이드 검색 결과를 함께 활용합니다.

4 실행 품질 점검

status, data_source, retrieval_source, fallback_used, error 값을 기준으로 실행 결과를 검토합니다.

5 장애 대응 흐름 정리

원인 후보, 조치 방안, 추가 확인 사항을 도출하는 Agent 처리 흐름을 확인합니다.

Day5는 개별 기능을 따로 확인하는 단계가 아니라, 제조 장애 대응 시나리오 안에서 MCP Tool 호출, DB 조회, RAG 검색, 실행 기록을 통합적으로 점검하는 단계입니다.

5.mcp_server_final
mcp_client_final

전체 개요

mcp_server_final과 mcp_client_final은 Day5 최종 실습에서 Server와 Client의 실행 경계를 확인하기 위한 예제입니다.

mcp_server_final은 제조 업무 Tool을 제공하는 MCP Server입니다. 설비 상태, 알람 이력, 품질 지표, 정비 이력 조회와 search_manual RAG 검색을 Tool로 노출하고, Tool 검증·실행·sanitize·logging을 담당합니다.

mcp_client_final은 MCP Server가 제공하는 Tool을 발견하고 호출하는 MCP Client입니다. 사용자 질문을 분석해 필요한 Tool을 선택하고, DB 조회 결과와 RAG 검색 결과를 비교·검토하여 화면에 보여 줍니다.

여기서 standalone 실행은 Server를 먼저 별도 프로세스로 실행하고, Client가 transport를 통해 Server에 연결하는 방식입니다. 이 방식은 Client가 Server 함수를 직접 import하지 않고, MCP 호출 경계를 통해 Tool을 사용하는 구조를 확인하기 위한 구성입니다.

mcp_server_final 설명

mcp_server_final은 Day5 최종 단계의 MCP Server입니다. FastMCP 기반으로 동작하며, Client가 호출할 수 있는 MCP Tool을 외부에 노출합니다.

주요 MCP Tool은 다음과 같습니다.

Tool	역할
execute_query	사용자 질문을 받아 Tool 선택부터 실행까지 연결
select_tools_for_manufacturing_query	제조 질문에 필요한 Tool Plan 생성
validate_tool_plan	Tool Plan의 계약 준수 여부 검증
execute_selected_tools	검증된 Tool Plan 실행
call_mcp_tool	개별 업무 Tool 호출
list_mcp_tools	사용 가능한 Tool 목록 확인

업무 Tool은 call_mcp_tool을 통해 호출됩니다. 예를 들어 설비 개요 조회, 최근 알람 이력 조회, 품질 지표 조회, 정비 이력 조회, search_manual 문서 검색이 여기에 해당합니다.

DB 조회는 PostgreSQL 읽기 전용 구조를 사용합니다. LLM이 SQL을 직접 만들지 않고, 서버 내부의 고정 SQL 함수와 parameter binding을 통해 조회합니다. DB 연결에 실패하면 mock fallback 결과를 반환할 수 있으며, 이때 data_source, fallback_used 값으로 실제 DB 조회인지 fallback인지 구분합니다.

search_manual은 RAG 검색을 사용합니다. 검색 환경이 준비되지 않았거나 검색 과정에서 문제가 발생하면 mock 문서 fallback을 사용할 수 있습니다.

서버는 반환 결과를 그대로 내보내지 않고 sanitize 후 반환합니다. 문서 전문, 민감 필드, 내부 경로, 불필요한 상세값은 외부로 노출하지 않는 구조입니다.

mcp_client_final 설명

mcp_client_final은 Day5 최종 단계의 MCP Client입니다. Client는 Server 내부 DB나 RAG 구현에 직접 접근하지 않고, Server가 노출한 MCP Tool만 호출합니다.

사용자 질문이 입력되면 Client는 질문의 의도를 규칙 기반으로 분석합니다. 예를 들어 설비 ID, 알람 코드, 라인 ID, 품질 지표 요청 여부 등을 확인한 뒤 필요한 Tool과 입력값을 구성합니다.

실행 결과에서는 다음 항목을 확인할 수 있습니다.

확인 항목	의미
status	Tool 호출 결과 상태
data_source	DB 조회인지 mock fallback인지 구분
retrieval_source	RAG 검색 출처 또는 fallback 여부
fallback_used	fallback 사용 여부
evidence_level	근거 수준 판단
error	연결 실패, timeout, rejected 등 오류 정보

mcp_client_final은 실행 결과를 성공처럼 포장하지 않습니다. 서버 미기동, timeout, Tool 거부, fallback 발생을 각각 구분해 표시합니다.

화면 실행은 Streamlit 앱을 통해 제공됩니다. 다만 Streamlit은 Agent 로직 자체가 아니라, Tool 호출 흐름과 판단 결과를 확인하기 위한 교육용 화면입니다.

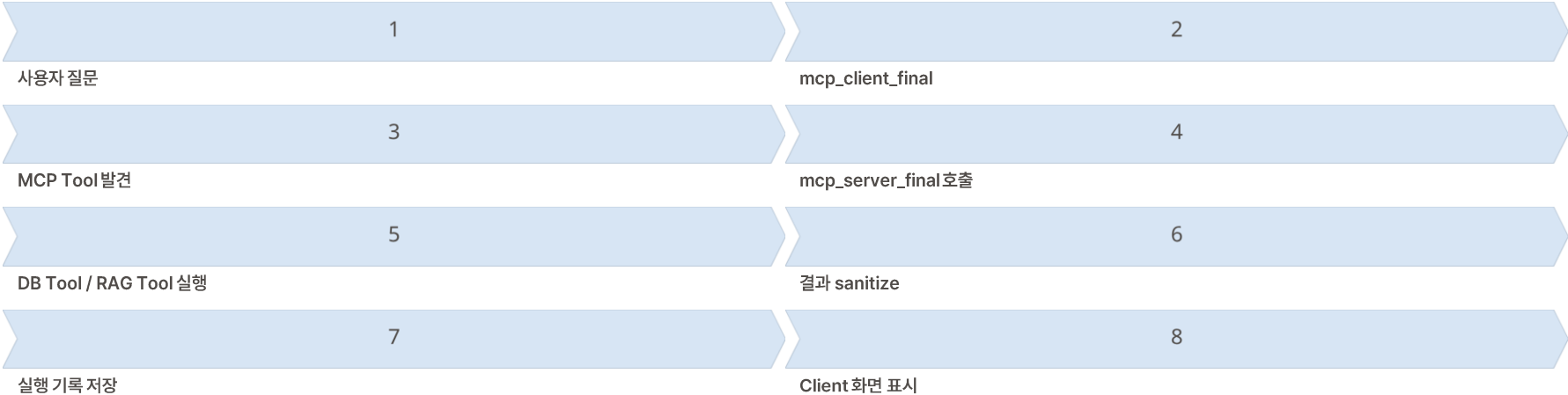
제조 장애 대응 Agent 통합 흐름

제조 장애 대응 Agent는 사용자 질문을 기준으로 필요한 Tool을 선택하고, DB 조회 결과와 RAG 검색 결과를 결합하여 대응 방향을 정리합니다.

통합 처리 단계

01	02	03
사용자 장애 질문 입력	Client가 질문 의도 분석	MCP Server의 Tool 목록 확인
04	05	06
설비 상태, 알람 이력, 품질 지표, 정비 이력 Tool 호출	search_manual로 기술 가이드 및 매뉴얼 검색	DB 조회 결과와 RAG 검색 결과 비교
07	08	09
원인 후보, 조치 방안, 추가 확인 사항 도출	fallback, timeout, error, evidence_level 확인	JSONL 로그에서 실행 기록 확인

핵심 흐름



Client는 판단 및 호출 흐름을 담당하고, Server는 Tool 제공, 검증, 실행, 결과 보호, 실행 기록을 담당하여 견고한 제조 장애 대응 시스템을 구축합니다.

가상 제조 장애 대응 실습 시나리오

최종 실습에서는 제조 장애 질문에 대한 DB 조회와 RAG 검색이 통합적으로 호출되어 Agent가 문제 해결을 위한 최적의 답변을 도출하는 과정을 확인합니다.

실습 질문 예시

EQ-03 설비에서 최근 알람이 반복되는데 원인 후보와 조치 방안을 알려줘

확인할 DB Tool

- 설비 상태 조회
- 최근 알람 이력 조회
- 품질 지표 조회
- 정비 이력 조회

확인할 결과

- 원인 후보
- 관련 매뉴얼 근거
- 조치 방안
- 추가 확인 사항

확인할 기록: JSONL 실행 로그

실습 순서

Server 실행

mcp_server_final을 HTTP방식으로 먼저 실행합니다.

Transport 설정

Client에서 transport를 HTTP로 선택하고 MCP HTTP URL을 서버 주소와 동일하게 설정합니다.

결과 확인

DB 조회 결과와 RAG 검색 결과가 함께 표시되는지 확인합니다.

확인할 RAG Tool

- search_manual

확인할 실행 정보

- status
- data_source
- retrieval_source
- fallback_used
- error

Client 실행

mcp_client_final Streamlit 화면을 실행합니다.

질문 입력

제조 장애 질문(EQ-03 설비에서 최근 알람이 반복되는데 원인 후보와 조치 방안을 알려줘)을 입력합니다.

오류 처리 확인

fallback, timeout, error가 정상 결과와 구분되어 표시되는지 확인합니다.

이 실습의 목적은 답변 문장뿐만 아니라, Agent가 어떤 근거를 조회했고 어떤 실행 경로를 거쳤는지 통합적으로 확인하는 데 있습니다.

대응 리포트 출력 구조

최종 Agent의 답변은 단순 문장 응답이 아니라, 조회 결과와 문서 근거를 바탕으로 대응 리포트 형태로 정리되어야 합니다.

리포트 구성 항목

장애 요약 사용자 질문, 설비 ID, 알람 코드, 증상 요약	DB 조회 결과 설비 상태, 알람 이력, 품질 지표, 정비 이력	문서 근거 search_manual 검색 결과의 snippet, doc_name, section_title, alarm_code
원인 후보 DB 결과와 문서 근거를 바탕으로 가능한 원인 정리	조치 방안 기술 가이드 또는 장애 대응 매뉴얼 기반의 점검-조치 방향	추가 확인 사항 근거가 부족하거나 추가 조치가 필요한 항목
실행 상태 status, fallback_used, data_source, retrieval_source, err or		

출력 예시 구조

원인 후보 최근 알람 반복과 품질 지표 변동을 함께 고려할 때 센서 이상 또는 설비 조건 변화 가능성	조치 방안 관련 매뉴얼 section 기준으로 센서 상태, 최근 정비 이력, 공정 조건을 순서대로 확인
추가 확인 사항 검색 결과가 부족한 경우 추가 로그 조회 또는 현장 점검 필요	근거 수준 DB와 RAG 결과가 모두 있는 경우 evidence_level을 높게 판단

대응 리포트는 원인을 확정하는 문서가 아니라, 조회 근거를 기반으로 원인 후보와 다음 확인 방향을 정리하는 결과물입니다.

최종 아키텍처 리뷰

Day5 최종 구조는 Client, MCP transport, Server, DB Tool, RAG Tool, 실행 기록 계층으로 나누어 검토합니다.

구성요소별 역할

Streamlit UI

사용자 질문 입력과 실행 결과 확인

mcp_client_final

질문 의도 분석, Tool 발견, Tool 호출, 결과 비교

MCP transport

Client와 Server 사이의 표준 호출 경계

mcp_server_final

Tool 제공, Tool 검증, Tool 실행, sanitize, logging

DB Tool

설비 상태, 알람 이력, 품질 지표, 정비 이력 조회

RAG Tool

기술 가이드와 장애 대응 매뉴얼 검색

Guardrail

Tool Contract 검증, 금지 Tool 차단, 입력값 제한

Trace / Log

실행 이력, 오류 원인, fallback 여부 추적

아키텍처 리뷰 질문



역할 분리

Client와 Server의 책임이 분리되어 있는가?



호출 경계

Client가 Server 내부 함수를 직접 import하지 않는가?



Tool 계약

허용 Tool과 입력 인자가 계약 기준으로 검증되는가?



데이터 접근

DB와 RAG 접근이 Server 내부 Tool을 통해서만 수행되는가?



보안 처리

반환 결과가 sanitize 후 외부로 전달되는가?



추적 가능성

실행 결과와 오류가 JSONL 로그로 남는가?

최종 아키텍처의 핵심은 Agent가 모든 것을 직접 처리하지 않고, MCP Server가 제공하는 검증된 Tool 경계를 통해 업무 기능을 사용하는 구조입니다.

실행 전 준비 사항

준비 항목	설명	확인 방법
작업 위치	프로젝트 루트에서 실행	예: c:\work\manufacturing_agent_project
PYTHONPATH	실행할 때 필요	\$env:PYTHONPATH = "src"
Python / uv 환경	uv 환경과 필요한 패키지 필요	.venv, uv, fastmcp 설치 상태 확인
DB 설정	PostgreSQL 연결 정보 필요	POSTGRES_HOST, POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
RAG 설정	Chroma, 임베딩, Ollama 설정 필요 가능	RAG 환경이 없으면 fallback 결과 확인
서버 host / port / path	HTTP 서버 기본값 확인 필요	기본 예: 127.0.0.1:8003/mcp
로그 위치	서버 실행 로그 저장 위치	outputs/day5/mcp_server_final/logs/mcp_server_final.jsonl

주의할 점은 서버를 `python -m day5...` 형태로 실행할 때 `day5` 패키지가 `src` 아래에 있으므로 `PYTHONPATH=src` 설정이 필요하다는 점입니다.

주요 트러블슈팅 체크리스트

최종 실습에서는 정상 실행뿐 아니라 연결 실패, fallback, timeout, Tool 거부 상황도 함께 확인해야 합니다.

Server 연결 실패

확인 항목: 서버 실행 여부, host, port, path

대응 방향: Client URL을 Server endpoint와 일치

Tool 목록 조회 실패

확인 항목: list_mcp_tools 결과

대응 방향: Server transport 설정과 실행 로그 확인

DB 조회 실패

확인 항목: data_source, fallback_used, error

대응 방향: PostgreSQL 설정 또는 mock fallback 여부 확인

RAG 검색 실패

확인 항목: retrieval_source, fallback_used, error

대응 방향: Chroma, Ollama 설정 또는 fallback 여부 확인

검색 결과 0건

확인 항목: documents 개수, evidence_level

대응 방향: 기술적 실패와 결과 없음 구분

Timeout 발생

확인 항목: timeout 표시, 서버 로그

대응 방향: HTTP long-lived 서버 실행 여부 확인

Tool 거부

확인 항목: validation 결과, rejected 이유

대응 방향: Tool Contract와 입력 인자 확인

응답 형식 문제

확인 항목: status, error, JSON 구조

대응 방향: sanitize 이후 반환 필드 확인

실습에서 중요한 것은 오류를 숨기지 않는 것입니다. 정상 결과, fallback 결과, 실패 결과를 구분해서 표시해야 실행 품질을 점검할 수 있습니다.

사내 적용 시 변경 요소

교육용 예제를 사내 시스템 적용 관점으로 확장할 때는 데이터, Tool Contract, 배포 환경, 로그 체계, 보안 기준을 함께 재설계해야 합니다.

교육용 예제와 사내 적용 변경 요소 비교

구분	교육용 예제	사내 적용 변경 요소
데이터	mock 제조 데이터	사내 DB, 로그 시스템, 설비 데이터 연계
검색	교육용 Chroma DB	사내 문서 저장소 또는 Vector DB 연계
Tool 정의	예제 Tool Contract	업무별 Tool 권한, 입력값, 출력값 기준 재정의
UI	Streamlit 화면	내부 업무 포털 또는 운영 UI 연계
로그	JSONL 로그	사내 관제, 감사, 운영 로그 체계 연계
오류 처리	fallback 처리	업무별 대체 처리 기준과 사용자 안내 기준 수립
실행 환경	local 실행	온프레미스 또는 폐쇄망 배포 구조 검토
시나리오	단일 장애 시나리오	설비, 품질, 정비, 공정별 시나리오 확장

사내 적용 검토 질문

데이터 어떤 DB, 로그, 문서를 Agent와 연결할 것인가?	권한 어떤 Tool을 어떤 사용자에게 허용할 것인가?
보안 내부 데이터가 외부 응답에 과도하게 포함되지 않는가?	운영 장애 발생 시 로그와 실행 이력을 추적할 수 있는가?
성능 RAG 검색, DB 조회, Tool 호출 지연을 어떻게 줄일 것인가?	배포 온프레미스 또는 보안 제약 환경에서 어떤 구성으로 배포할 것인가?

사내 적용의 핵심은 교육용 구조를 그대로 옮기는 것이 아니라, 업무 데이터, 권한, 보안, 운영 로그 기준에 맞게 Tool 경계와 실행 흐름을 재설계하는 것입니다.

6.mcp_server_final

mcp_client_final

실행

서버 standalone 실행 방법

HTTP 방식은 서버를 먼저 long-lived 프로세스로 실행하고, Client가 해당 서버 URL로 접속하는 방식입니다.

```
# 터미널 1: 프로젝트 루트에서 실행
```

```
$env:PYTHONPATH = "src"
```

```
uv run python -m day5.mcp_server_final.mcp_server --transport streamable-http --host 127.0.0.1 --port 8003 --
```

HTTP endpoint는 다음과 같습니다.

```
http://127.0.0.1:8003/mcp
```

HTTP 방식은 서버 프로세스가 계속 유지되므로 RAG 검색이나 DB 조회처럼 초기화 비용이 있는 흐름을 확인하기에 적합합니다.

서버가 정상 실행되면 터미널 1은 대기 상태가 됩니다. Client는 별도 터미널에서 실행해야 합니다.

mcp_server_final 실행

```
관리자: Windows PowerShell
PS C:\WINDOWS\system32> cd C:\work\manufacturing_agent_project\W
PS C:\work\manufacturing_agent_project>
PS C:\work\manufacturing_agent_project> $env:PYTHONPATH = "src"
PS C:\work\manufacturing_agent_project>
PS C:\work\manufacturing_agent_project> uv run python -m day5.mcp_server_final.mcp_server --transport streamable-http --
host 127.0.0.1 --port 8003 --path /mcp
[06/04/26 05:08:07] INFO      Starting MCP server 'mcp_server_final' with transport 'streamable-http' on transport.py:301
                             http://127.0.0.1:8003/mcp
INFO:      Started server process [8796]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8003 (Press CTRL+C to quit)
```

client에서 standalone 서버에 연결하는 방법

먼저 터미널 1에서 mcp_server_final HTTP 서버를 실행합니다. 그 다음 터미널 2에서 mcp_client_final Streamlit 앱을 실행합니다.

```
# 터미널 2: 프로젝트 루트에서 실행
$env:PYTHONPATH = "src"
# 터미널 2: 프로젝트 루트에서 실행
uv run streamlit run src/day5/mcp_client_final/streamlit_app.py
```

화면에서 transport를 http로 선택하고 MCP HTTP URL을 서버 주소에 맞추습니다.

```
http://127.0.0.1:8003/mcp
```

client에서 standalone 서버에 연결하는 방법

```
관리자: Windows PowerShell
PS C:\work\manufacturing_agent_project> cd C:\work\manufacturing_agent_project
PS C:\work\manufacturing_agent_project>
PS C:\work\manufacturing_agent_project> $env:PYTHONPATH = "src"
PS C:\work\manufacturing_agent_project>
PS C:\work\manufacturing_agent_project> uv run streamlit run src/day5/mcp_client_final/streamlit_app.py
2026-06-04 06:14:52.804 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.71:8501
```

client 실행

mcp_client_final — Agent 판단 흐름 시각화

단순 조회/보고 UI가 아니라, Agent가 '의도 판단' → Tool 선택 → 호출 → 비교·검증 → 교육용 점검 등급 → 다음 확인 항목'을 어떻게 수행하는지 단계별로 보여 주는 교육용 예제입니다. (실행 호출은 mcp_client04의 ToolCaller/config를 재사용)

Agent 실행 다중 DB Tool 병렬 호출

1) 질문 입력

예시 질문(수업 시연용)

(직접 입력) ▼

(직접 입력)

01. DB 조회 - 설비 상태 — EQP-EV-03 지금 상태 알려줘

02. DB 조회 - 설비 개요 — EQP-EV-03 설비 개요 알려줘

03. DB 조회 - 최근 알람 이력 — EQP-EV-03 최근 알람 이력 보여줘

04. DB 조회 - 알람 코드 필터 — EQP-EV-03에서 ALM-TEMP-402 알람이 몇 번 발생했는지 알려줘

05. DB 조회 - 불량률 지표 — EQP-EV-03 불량률 지표 알려줘

06. DB 조회 - 라인 수율 지표 — LINE-07 수율 지표 보여줘

07. DB 조회 - 결선 경계 지표 — EQP-EV-03 결선 경계 지표 알려줘

Agent 판단 흐름 실행

mcp_client_final — Agent 판단 흐름 시각화

단순 조회/보고 UI가 아니라, Agent가 '의도 판단' → Tool 선택 → 호출 → 비교·검증 → 교육용 점검 등급 → 다음 확인 항목을 어떻게 수행하는지 단계별로 보여 주는 교육용 예제입니다. (실행 호출은 mcp_client04의 ToolCaller/config를 재사용)

Agent 실행 다중 DB Tool 병렬 호출

1) 질문 입력

예시 질문(수업 시연용)

02. DB 조회 - 설비 개요 — EQP-EV-03 설비 개요 알려줘

💡 설비 ID를 기준으로 설비 유형, 라인 정보, 데이터 출처가 어떻게 표시되는지 확인합니다. · 핵심: get_equipment_overview, equipment_type, line_id, data_source

질문

EQP-EV-03 설비 개요 알려줘

mcp_client_final은 standalone HTTP 서버 연결만 사용합니다. 먼저 mcp_server_final을 8003 포트로 실행한 뒤 Client를 실행해 주세요.

연결 대상 MCP HTTP endpoint(고정): <http://127.0.0.1:8003/mcp>

```
uv run python -m day5.mcp_server_final.mcp_server --transport streamable-http --host 127.0.0.1 --port 8003 --path /mcp
```

Agent 판단 흐름 실행

Windows 정품 인증

[설정]으로 이동하여 Windows를 정품 인증합니다.

1단계 · 질문 분석

equipment_id	alarm_code	line_id	metric_name
EQP-EV-03	-	-	-

추출 키워드: 상태(개요)

2단계 · 의도 분류

intent → DB_ONLY (needs_db=True, needs_rag=False)

이유: 설비 상태/알람/품질 등 DB 사실만 필요한 질문입니다.

3단계 · 선택된 MCP Tool / 이유 / 순서

순서	tool	목적	선택 이유
1	get_equipment_overview	설비 개요 확인	설비/라인 식별자가 있어 기본 개요(사실 데이터)를 확인합니다.

5단계 · DB 결과 요약

DB Tool 이 정상 응답했습니다.

data_source=['db'] · mock_fallback=False

6단계 · RAG 결과 요약

RAG 의도가 아니어서 search_manual 을 호출하지 않았습니다.

retrieval_source=None · fallback_used=None · documents=0

7단계 · DB↔RAG 비교 + Evidence Check ⇄

evidence_level: PARTIAL — DB 또는 RAG 중 한쪽만 근거가 있어 근거가 부분적입니다.

db_supported=True · rag_supported=False · mock_fallback=False

8단계 · 교육용 점검 등급 (점검 우선순위)

점검 우선순위: 낮음

상태 조회 위주이고 뚜렷한 이상 근거가 확인되지 않았습니다(교육용 기준). ※ 교육용 예제 기준이며 실제 운영 판단이 아닙니다.

※ 교육용 예제 기준이며 실제 운영 판단이 아닙니다.

9단계 · 다음 확인 항목 / 후속 질문

다음 확인 항목(추가 확인 방향 – 조치 명령 아님):

- EQP-EV-03 최근 알람 이력 추가 확인
- EQP-EV-03 정비 이력 확인
- 해당 설비/라인의 품질 지표 변화 확인
- 담당자가 실제 현장 상태를 직접 확인(교육용 예제는 판단을 대신하지 않음)

후속 질문 후보:

- EQP-EV-03의 최근 정비 이력도 확인할까요?
- 품질 지표 추세도 함께 볼까요?

▼ 원본 결과 dict 보기 (raw)

```
{
  "question": "EQP-EV-03 설비 개요 알려줘"
  "analysis": {
    "equipment_id": "EQP-EV-03"
    "alarm_code": NULL
    "line_id": NULL
    "metric_name": NULL
    "keywords": [
      0: {
        "category": "상태"
        "matched": [
          0: "개요"
        ]
      }
    ]
  }
  "raw": "EQP-EV-03 설비 개요 알려줘"
}
"intent": {
  "intent": "DB_ONLY"
  "reason": "설비 상태/알람/품질 등 DB 사실만 필요한 질문입니다."
  "extracted_entities": {
    "equipment_id": "EQP-EV-03"
    "alarm_code": NULL
    "line_id": NULL
    "metric_name": NULL
  }
  "needs_db": true
  "needs_rag": false
}
```

```
  0 : {
    "tool_name" : "get_equipment_overview"
    "purpose" : "설비 개요 확인"
    "reason" : "설비/라인 식별자가 있어 기본 개요(사실 데이터)를 확인합니다."
    "call_order" : 1
    "input_hint" : {
      "equipment_id" : "EQP-EV-03"
    }
  }
]

"tool_trace" : [
  0 : {
    "tool_name" : "get_equipment_overview"
    "status" : "OK"
    "data_source" : "db"
    "retrieval_source" : NULL
    "fallback_used" : false
    "mock_fallback" : false
    "error" : NULL
    "payload" : {
      "equipment_id" : "EQP-EV-03"
      "equipment_type" : "Evaporation Chamber"
      "line_id" : "LINE-07"
      "data_source" : "db"
      "fallback_used" : false
    }
  }
]
]
```

```
▼ "educational_check_level" : {  
  "check_level" : "낮음"  
  "reason" : "상태 조회 위주이고 뚜렷한 이상 근거가 확인되지 않았습니다(교육용 기준). ※ 교육용 예제 기준이며 실제 운영 판단이 아닙니다."  
  ▼ "supporting_tools" : [  
    0 : "get_equipment_overview"  
  ]  
  ▶ "uncertainty" : []  
}  
▼ "next_checks" : [  
  0 : "EQP-EV-03 최근 알람 이력 추가 확인"  
  1 : "EQP-EV-03 정비 이력 확인"  
  2 : "해당 설비/라인의 품질 지표 변화 확인"  
  3 : "담당자가 실제 현장 상태를 직접 확인(교육용 예제는 판단을 대신하지 않음)"  
]  
▼ "follow_up_questions" : [  
  0 : "EQP-EV-03의 최근 정비 이력도 확인할까요?"  
  1 : "품질 지표 추세도 함께 볼까요?"  
]  
"final_message" : "의도=DB_ONLY · 근거수준=PARTIAL · 교육용 점검 등급=낮음. 결과는 확정이 아니라 '확인·검토용'이며, 장비 제어 지시는 제공하지 않습니다(교육용)."  
}
```